

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

Lernziele

- Welche Arten von Datenbankmanagementsystemen existieren?
- Welche Komponenten werden in einem DBMS gebraucht?

Übersicht

- 1 Einführung
- Szenarien
- Begrifflichkeiten
- Informationssysteme

Datenbanken: Ausgangspunkt

- Daten über einen bestimmten Problembereich sollen **dauerhaft** gespeichert und für ihre **Verarbeitung** zur Verfügung stehen.
- Dauerhaft heißt: nach Verlassen von Programmen, die mit den Daten arbeiten, bleiben diese erhalten.
- Verarbeitung heißt:
 - Bearbeitung durch Datenverarbeitungsverfahren (Rechenverfahren, Programme) oder Verwendung in Geschäftsprozessen (durch einen/mehrere Anwender)
 - Wichtigster Verarbeitungsschritt: (Komplexe) Abfragen

Datenbanken: Ausgangspunkt (Anfang 60er Jahre)

- Dateibasierte Datenverwaltung:
- Daten werden als Dateien im Dateisystem gespeichert

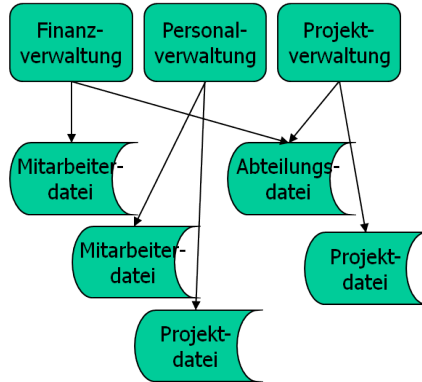


Abbildung: Zugriff auf Dateien ohne spezielle Verwaltung

Beispiel CSV-Datei

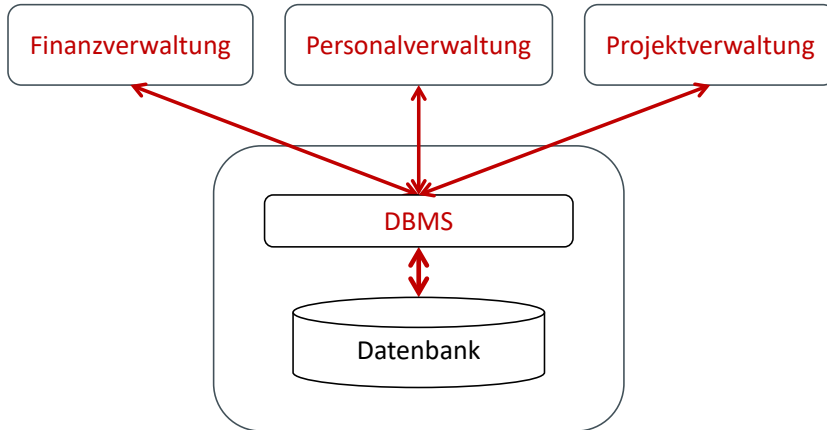
1	;	Manuel	;	Neuer	;	Torwart
5	;	Mats	;	Hummels	;	Abwehr
17	;	Per	;	Mertesacker	;	Abwehr
7	;	Bastian	;	Schweinsteiger	;	Mittelfeld
10	;	Julian	;	Draxxler	;	Mittelfeld
15	;	Lars	;	Bender	;	Mittelfeld
11	;	Miroslav	;	Klose	;	Sturm

Dateibasierte Datenverwaltung

- Vorteile?
 - Schnell programmiert, kostengünstig.
- Nachteile? (vgl. Kemper)
 - Redundanzen und Inkonsistenzen
 - Verschiedene Datenformate
 - Probleme beim Mehrbenutzerbetrieb
 - Verlust von Daten
 - Integritätsverletzungen
 - Sicherheitsprobleme
 - Hohe Entwicklungskosten für Anwendungsprogramme

Zweite Idee: DBMS

Datenverwaltung durch Einfügen einer Managementebene:
Datenbankmanagementsystem (DBMS)



Begriffe in Datenbanken

Datenbankmanagementsystem (DBMS)

- Programme bzw. Prozesse, die zur koordinierten, sicheren und performanten Verwaltung großer strukturierter Datenbestände dienen

Datenbank (DB)

- Strukturierter, von einem DBMS verwalteter Datenbestand

Datenbanksystem (DBS)

- Datenbankmanagementsystem (DBMS)
Software zur Verwaltung von Datenbanken
- Datenbank (DB)

Datenbankunterstützung

Klassisch für relationale Datenbanken



Datenbankbeispiele

Indirekt benutzen wir ständig Datenbanken:

- Websuchen in Suchmaschinen
- eShopping Seiten
- Backend größerer Webseiten (Content Management System)

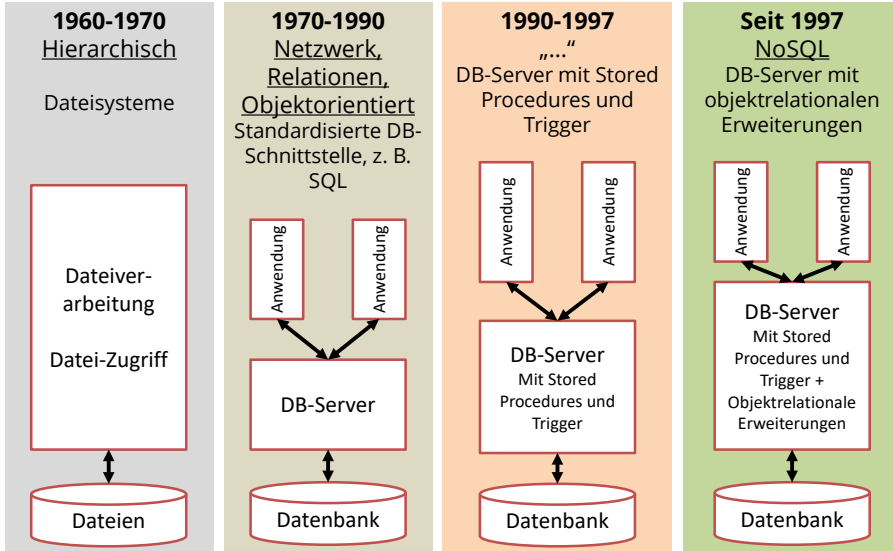
Datenbanken werden fast immer eingesetzt wenn ...

- die Datenmengen recht groß sind
- die Daten einen gewissen Wert besitzen

Übersicht

- 1 Einführung
 - Szenarien
 - Begrifflichkeiten
 - Informationssysteme

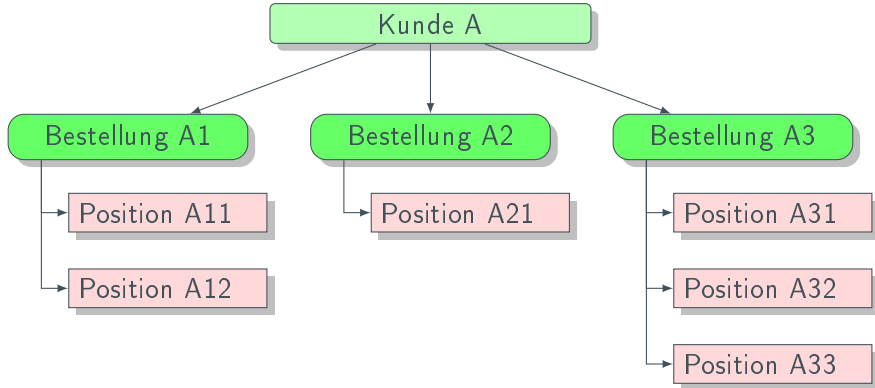
Historie von Datenbankmodellen



Hierarchische Datenbanken (60 und 70er)

- Hierarchische Datenbanken (60/70er)
- Daten werden in einer Baumstruktur hierarchisch abgelegt (mit internen Knoten und Blättern)
- Jeder Knoten enthält Datensätze (Records)
- Jeder Knoten hat nur einen Vorgänger (außer Wurzel) und beliebig viele Nachfolger (außer Blätter)
- Struktur wird von DBMS überwacht
- Immer noch strukturabhängig: Zugriff auf Daten nur bei Kenntnis des Pfades durch den Baum
- Immer noch Redundanz, da nur 1 : 1 und 1 : n Beziehungen möglich, $n : m$ nur durch Redundanz möglich.

Hierarchische Datenbanken (60/70er)

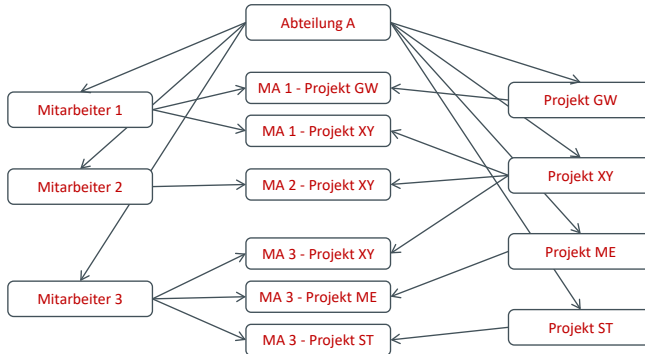


Hierarchisches Modell

Netzwerkmodell (70er)

- Weiterentwicklung des hierarchischen Modells
- Vorschlag der *Conference on Data Systems Languages* (CODASYL) im Jahr 1971
- Einführung von Subschemas (Views bzw. Sichten)
- Entwurf einheitlicher Datenbanksprachen (Schema Data Description Language, Subschema Data Description Language, Data Manipulation Language)
- Aufhebung der Beschränkung nur eines Vorgängers (m:n-Beziehungen werden möglich)
- Resultat: Der Baum wird zum Netz.
- Problem: Keine Strukturunabhängigkeit beim Zugriff auf Daten

Netzwerkmodell (70er)



Relationale Datenbanken

- Relationales Modell (Ende 70er)
- Erfinder: Dr. E.F. Codd (Mathematiker bei IBM)
- Systeme
 - 1978 SystemR von IBM
 - 1977 Oracle von Relational Software (später Oracle)
 - 197x Ingres von University of California, Berkeley (Quellcode unter BSD-Lizenz, Nachfolger Postgres heißt heute PostgreSQL und ist OpenSource)
- Folgt ANSI-SPARC: Daten- und Strukturunabhängigkeit
- Vollständig mathematisch beschrieben durch Relationenalgebra
- Wichtige DB-Sprache: **SQL** (Structured Query Language)

Weit verbreitet: Relationale Datenbanken

Arztpraxis

Bezeichnung	Ort	BS-Nr.
HNO Huber	Lage	3584952
Gyn Meyer	Detmold	4858458
...	...	...

Arzt

AID	Name	Vorname	Wohnort	BS-Nr.
4711	Vinus	Petra	Brakel	3584952
4713	Höpfner	Vitali	Lemgo	4858458
...	...	...	...	...

behandelt

AID	PID
4711	1346
4711	3164
...	...

Patient

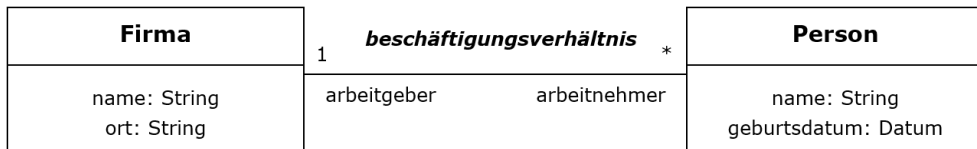
PID	Name	Vorname	Geschlecht
1346	Huber	Birgi	W
3164	Seefeld	Dietmar	M
...	...	...	...

Objektorientierte Datenbanken

- Objektorientierte Datenbanken (Mitte der 80er)
- Problem Relationaler DBS: Trennung von Daten und Funktion
- Idee (am Bsp.): Warum soll ein DB-Eintrag zu einer Person, über die das Geburtsdatum gespeichert ist, nicht auch in der Lage sein, das Alter in Jahre auszurechnen?
- Vorteil: „Objekt“ nahe Funktionen sind auch bei dem Objekt selbst zu finden, und nicht verteilt in verschiedenen Programmen (wo immer es nötig ist, das Alter auszurechnen) => Änderungsfreundlichkeit
- Parallel: Aufkommen objektorientierter Programmiersprachen mit Bedarf, Objekte zu speichern

Objektorientierte Datenbanken

Klassen und Beziehungen:



Instanziierung der Beziehung (grafisch):

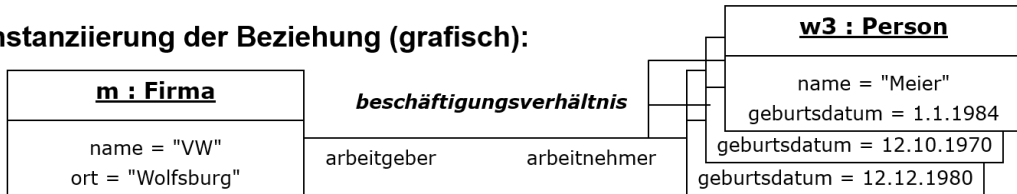
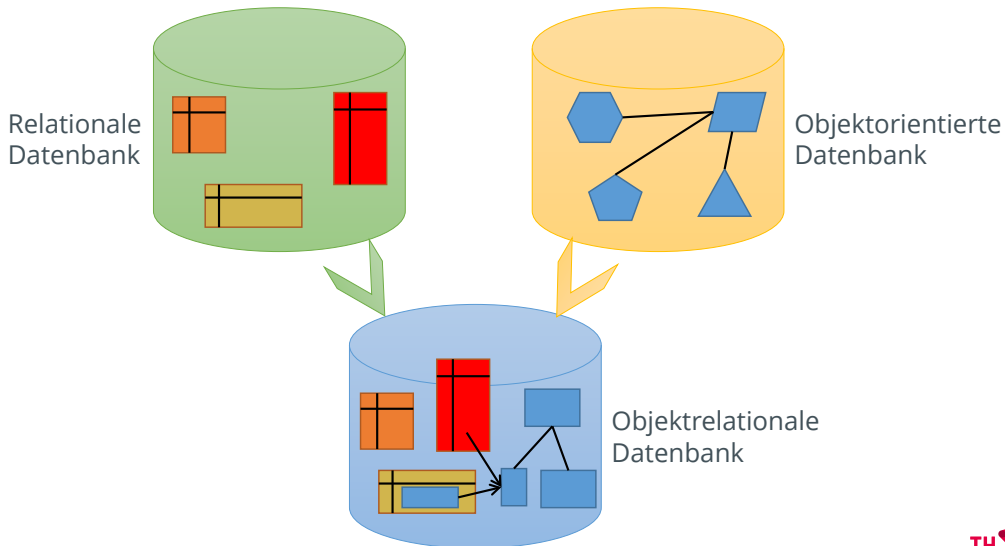


Abbildung: Klassen und zugehörige Instanzen einer objektorientierten DB

Objektrelationale Datenbanken



Objektrelationale Datenbanken

- Probleme objektorientierter DBMS:
 - Keine mit SQL vergleichbare, standardisierte Datenbanksprache
 - Teilweise langsamer als relationale DBMS
- Kompromiss: Objektrelationale DBMS: Erweiterung des relationalen Modells um objektorientierte Konzepte oft durch Aufsatz einer objektorientierten Zugriffsschicht
- Beibehaltung/Erweiterung der SQL-Sprache
- Beispiele:
 - PostgreSQL
 - Oracle

XML-Datenbanken

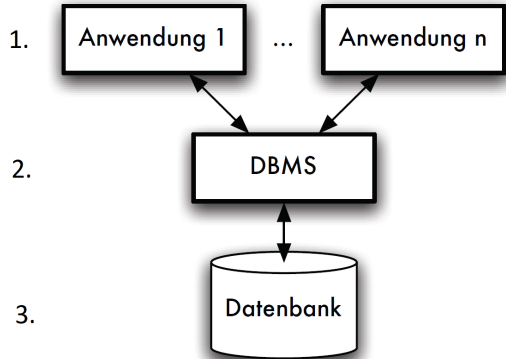
- XML: DER Standard für die Darstellung strukturierter Daten in elektronischen Dokumenten
- Von wem? W3C
- Prinzip: Text & Markups
- Speicherkonzept basiert auf XML-Dateien
- Zwei Kategorien:
 - **XML-enabled** (datenorientierter Ansatz): Herkömmliche DBMS (z. B. rel. DBMS) die ein Mapping auf oder ins XML-Format erlauben.
 - **Native XML-Datenbanksysteme** (dokumentenorientierter Ansatz): Diese Systeme speichern die Information, ähnlich wie bei der Speicherung von XML-Dokumenten im Dateisystem, direkt als XML-Dokumente ab.

Beispiele und Links zu DBMS

- Oracle Database (www.oracle.com/de/database/)
- MS SQL-Server (www.microsoft.com/de-de/sql-server/)
- IBM Db2 (www.ibm.com/software/data/db2/)
- MySQL & MariaDB (www.mysql.de & www.mariadb.com)
- Firebird (<http://www.firebirdsql.org/>)
- PostgreSQL (<http://www.postgres.de/>)
- eXistDB (<http://exist-db.org/>)
- Apache Xindice (xml.apache.org/xindice/)
- Apache CouchDB (<http://couchdb.apache.org/>)
- MongoDB (www.mongodb.com/)
- neo4j (<https://neo4j.com>)

3 Schichten Modell (ANSI-SPARC - Architektur)

- 1 **Externe Modelle:** Sicht von Benutzer(gruppen)/Anwendungsprogrammen
- 2 **Konzeptuelles Modell:** Logische Ebene; Zugriff von Benutzern/Anwendungen auf darunterliegenden Daten erfolgt ausschließlich über das DBMS; fungiert als Manager, Filter.
- 3 **Internes Modell:** Physische Speicherstrukturen, welche die „Datenbank“ ausmachen. Früher befanden sich auf dieser Ebene die Dateien.



3 Schichten Modell

Zielsetzung: Datenunabhängigkeit:

- Physische Datenunabhängigkeit
- Internes Schema kann geändert werden, ohne Anwendungsprogramme zu ändern
- Änderung nur bei Abbildung konzeptuelles Schema (internes Schema)
- Logische Datenunabhängigkeit
- Konzeptuelles Schema kann geändert werden ohne Anwendungsprogramme zu ändern solange das entsprechende externe Modell abgeleitet werden kann

Verwendung von DBMS

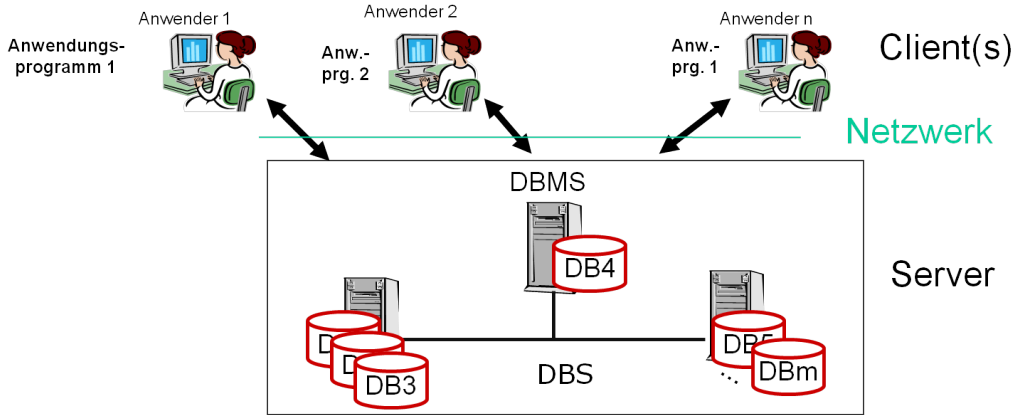
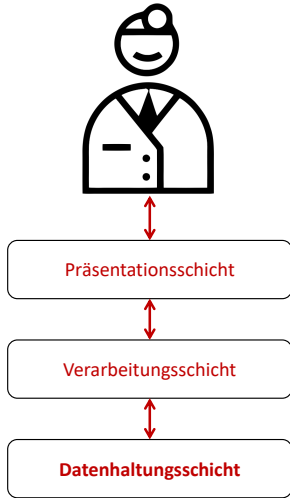


Abbildung: Client-Server-Architektur

Drei-Schichten-Architektur für Anwendungsprogramme



Grafische Benutzerschnittstelle (z. B. Eingabefenster)

Programmlogik (unabhängig von der Benutzerschnittstelle); Funktion

Daten der Anwendung (z. B. in einer Datenbank)

Datenbank(abfrage)sprachen

- Definition Datenbank(abfrage)sprache:
- Zentrales Element eines DBMS und spezielle Computersprache zur
 - Erstellung und Modifikation von DBen,
 - Speicherung, Abfrage und Manipulation von Daten
- Beispiele:
 - RDBMS (Relationales DBMS):
 - **SQL** (Structured Query Language)
 - **QBE** (Query by Example)
 - XML-DBMS
 - **XQL** (XML Query Language)
 - Graphen-Datenbanken
 - **GQL** (Graph Query Language)
 - **SPARQL** (SPARQL Protocol And RDF¹ Query Language)

¹RDF = Resource Description Framework

Datenbank(abfrage)sprachen

- Unterteilung Datenbank(abfrage)sprache:
 - **DML** (Data Manipulation Language: „Datenverarbeitungssprache“): Sprache/Sprachteile für das Abfragen, Einfügen, Ändern oder Löschen von Nutzdaten
 - **DDL** (Data Definition Language: „Datenbeschreibungssprache“): Sprache/Sprachteile für das Anlegen, Ändern und Löschen von Datenstrukturen
 - **DCL** (Data Control Language: „Datenaufsichtssprache“): Sprache/Sprachteile für die Zugriffskontrolle
- Beispiel:
 - **SQL** enthält **DML**, **DDL** und **DCL**
 - **QBE** enthält **DML**
 - **XQL** enthält u. a. XPath

Übersicht

- 1 Einführung
 - Szenarien
 - Begrifflichkeiten
 - Informationssysteme

Aufgaben und Funktionen eines DBMS nach Codd (1)

- **Integration**
Hier verbirgt sich die Möglichkeit der kontrollierten nicht-redundanten Datenhaltung des gesamten relevanten Datenbestands
- **Operationen**
Zugriff, Manipulation auf Daten und Management der DB
- **Katalog** („Data Dictionary“)
Spezifikation aller Inhalte der Datenbank(en)
- **Benutzersichten**
Bedarfsgerechte Darstellung relevanter Datensätze
- **Konsistenzüberwachung** („Integritätssicherung“)
Gewährleistung korrekter Datenbankinhalte und der korrekten Ausführung von Änderungen, so dass diese die Konsistenz nicht verletzen können

Aufgaben und Funktionen eines DBMS nach Codd (2)

- **Zugriffskontrolle**
Benutzer, Rechte, Rollen
- **Transaktionen**
Zusammenfassung von Datenbankänderungen zu Funktionseinheiten, die als Ganzes ausgeführt werden sollen und die bei Erfolg permanent in der Datenbank gespeichert werden.
- **Synchronisation**
Behandlung konkurrierender Transaktionen (Schreiben, Aktualisieren, Löschen)
- **Datensicherung**
Backup, Wiederherstellung

Aufbau und Organisation einer Datenbank

- ANSI-SPARC: Eine Datenbank besteht aus
 - ① Festlegungen, wie die Daten organisiert sind (Metadaten, DBMS: Datenbankschema),
 - ② Festlegung wichtiger „Sichten“ auf die Daten,
 - ③ den Daten selbst, die dem Datenbankschema entsprechend strukturiert sind.
- Der Festlegung eines Datenbankschemas geht im Rahmen der Datenbankentwicklung der Entwurf eines (konzeptuellen/semantischen) Datenmodells voraus.
- Hierfür gibt es eigene Methoden, Sprachen, Werkzeuge (z. B. ER-Modellierung)

Wichtige Komponenten des DBMS

1 Data-Dictionary/Repositories

- Data-Dictionary (Datenlexikon,-wörterbuch; auch Meta²-Datenbank oder Katalog): Speichert Informationen ÜBER die Datenbank:
 - Datenbankschema (Tabellen, Beziehungen, etc.)
 - Zugriffsrechte
- Repository
 - Erweiterung des Data-Dictionaries um Informationen über Anwendungsprogramme/ -funktionen, Benutzer, etc.

2 Logbuch

- Speichert alle Transaktionsvorgänge (Beginn und Endzeitpunkt, Zustand der DB vor Beginn der Transaktion, ...)
- Wird für das Zurücksetzen der Datenbank im Fehlerfalle benötigt

²Metadaten: Daten über Daten. Beispiel für Adresse: Straße(string), Hausnummer (string->wegen 12b), PLZ(string->Landesabhängig auch Buchstaben mgl.), Ort(string)

Wichtige Komponenten des DBMS

3 Administrationswerkzeuge

- Programme zur Administration des DBMS (Einrichten einer Datenbank, Zugriffsrechteverwaltung etc.)
- Bsp.: MySQL Administrator

4 Entwurfswerkzeuge

- Programme zum Entwurf der Datenbank unter „Umgehung“ der DDL (oft grafisch-interaktiv; z. B: ER-basiert)
- Bsp.: MySQL Workbench, DBeaver, ...

5 Abfrage-Generatoren

- Programme zum Erstellen von Abfragen unter „Umgehung“ der DQL/DML
- Bsp.: MySQL Workbench, DBeaver, Microsoft-Access mit den QBE-Ansatz, Oxygen XML Editor für XML-DBMS

Wichtige Komponenten des DBMS

⑥ Report-Generatoren

- Programme zum Erzeugen von Berichten über Datenbankinhalte in verschiedensten Formen
- Bsp.: MS-Access (Berichte über Tabellen, Beziehungen, Tabelleninhalte)

⑦ CASE-Werkzeuge (Computer Aided Software Engineering)

- Entwicklungsumgebung zur Erstellung komplexer DB-Anwendungen mit z. T. automatisierter Sourcecode-Generierung)
- Bsp.: MS-Access, MS Visual Studio, Eclipse, NetBeans

⑧ Werkzeuge zur Fehleranalyse

⑨ Funktionen zur Komprimierung und Reorganisation der DB

⑩ Archivierungsfunktionen

Personen und Rollen

- Datenbankadministrator (**DBA**)
 - Verwaltet die Ressource Datenbank
 - Internes Schema
 - Vergabe von Zutrittsrechten
 - Tuning, Performance und Monitoring
 - Sicherheit und Zuverlässigkeit (Backups und Recovery)
- Unternehmensadministrator (**Datenbankdesigner**)
 - Zuständig für konzeptuelles Schema und externe Schemata
 - Schnittstelle zur Software-Entwicklung

Personen und Rollen

- Systemanalytiker, Anwendungsprogrammierer
 - Anforderungserhebung
 - Software-Entwicklung
- Endbenutzer
 - Gelegentliche Benutzer z. B. Manager
 - Unterschiedliche, z. T. nicht vorhersehbare Informationsbedürfnisse
 - Von „schnell mal nachschauen“
 - Bis komplexe Analysen
- Parametrische Benutzer z. B. Sachbearbeiter
 - Anwendungsprogramme, „canned transactions“
- Power-User z. B. Analytiker
 - Komplexe Anforderungen
 - Gute Kenntnis von DB & Schnittstellen

Vielen Dank!